

Full Stack Developer: Role & Technical Experience

Overview

In my previous role, I worked as a Full Stack Developer on an indoor localisation platform designed for construction sites. The platform used indoor localisation devices to provide managers, supervisors, and foremen with visibility into how work was progressing on site. The system collected location and sensor data from devices deployed throughout construction sites and transformed that data into information that could be used to monitor daily site activity and work efficiency. I initially joined the team as a Frontend Developer, primarily working with Angular. As I became more involved with the system, my responsibilities expanded across the backend, database, cloud infrastructure, and deployment architecture.

Frontend Development

I initially focused on developing the main web dashboard using **Angular**. The dashboard was the primary interface through which customers interacted with the platform. It provided visualisations of construction site activity, time series data, and indoor locations.

Angular Architecture

I structured the Angular application using a modular architecture to make the application easier to maintain and scale as more functionality was introduced. The application was divided into multiple components, with functionality organised into separate modules and routing modules. Shared functionality was placed in a dedicated shared area so that reusable components could be used throughout the application without duplicating code.

API communication was centralised through Angular services, keeping HTTP communication separate from the presentation layer. This structure allowed new features and sections of the dashboard to be added without turning the application into a single tightly coupled codebase.

Data Visualisation

The dashboard used several visualisation libraries to present large amounts of operational data in a way that could be understood by users working on construction sites. I worked with different charting and visualisation libraries to display timeseries and operational information. I also integrated Mapbox into the dashboard to visualise indoor positioning data and construction site floor plans.

Custom Map Tile Infrastructure

One challenge was displaying indoor floor plans through **Mapbox** without relying on expensive external map data services for information that was specific to each construction site. To solve this, we built our own tileserving infrastructure. Floor plan

data was stored in **AWS S3**, and we developed a tile server approach that allowed the dashboard to retrieve and display the required indoor map tiles through Mapbox. This allowed us to use Mapbox's mapping capabilities while maintaining control over the site specific map data.

Authentication and Authorization

I integrated **Keycloak** into the Angular dashboard to provide authentication and role based access control (RBAC), ensuring users could access functionality and data according to their responsibilities. We deployed and managed our own Keycloak service within our Kubernetes (K8s) cluster, giving us control over the authentication infrastructure and its configuration.

We designed a multi tenant realm architecture to support different organisations, with structured user, role, and permission management to maintain security and tenant isolation. Role based permissions ensured that users could only access the resources and functionality relevant to their responsibilities.

Authorisation was enforced at both the frontend and backend levels. While the Angular dashboard controlled the visibility of pages and features based on user roles, the backend independently protected API endpoints, ensuring that only authenticated users with the appropriate permissions could access specific resources or perform particular operations. This established a secure and maintainable authentication and authorisation system for a multi tenant platform.

Backend Development

The backend was primarily built using **Node.js** and consisted of two major services:

- API Gateway
- Data Import Service

API Gateway

The API Gateway acted as the main backend interface for the dashboard. It was responsible for receiving API requests, determining what information was required, executing the appropriate database queries, and returning the required data to the frontend. A significant part of my work involved improving database queries and the way large amounts of time series data were retrieved. The system needed to retrieve historical data over relatively large periods while maintaining acceptable response times. I worked on query optimisation and data retrieval logic so that large datasets could be processed efficiently.

Data Import Service

The Data Import service was responsible for receiving data generated by the indoor localisation devices. The devices communicated through a third party platform, The Things Stack, which handled the communication layer for our IoT devices. The data import pipeline evolved during the project. Initially, we used webhooks to receive device messages. Later, we moved to **MQTT**, which provided a more suitable communication mechanism for our use case. The Node.js service was responsible for:

1. Receiving incoming device messages.
2. Parsing the messages.
3. Processing the relevant information.
4. Transforming the data into the required structure.
5. Writing the processed information into **PostgreSQL**.

Each customer had a dedicated instance of the Data Import service running within their Kubernetes namespace.

Database migrations were integrated into the Data Import service. This allowed a new customer's database structure to be established quickly when deploying a new customer environment, reducing the amount of manual database configuration required during deployment.

Multi Tenant Kubernetes Architecture

One of the more interesting parts of the system was how customer environments were isolated. Our application ran on a **Kubernetes cluster**, with a separate namespace for each customer. Each namespace contained the services required for that customer's environment, including:

- Frontend/dashboard
- API backend
- Data Import service
- Scheduled jobs, such as weekly database backups stored as dump files in an S3 bucket
- Other customer specific services

This approach provided logical separation between customers while allowing us to operate the platform using a shared Kubernetes infrastructure. Each customer also had their own PostgreSQL database hosted on AWS RDS.

In addition to developing and deploying services, I managed the Kubernetes cluster, monitored its health, and ensured that workloads were running smoothly. This involved checking logs, investigating errors, and identifying and troubleshooting failed or non-running containers. I also deployed and configured new customer environments, setting up the required namespaces and services to onboard new customers.

Customer Configuration and Secrets

Because each customer had an isolated environment and database, the services needed to know which database and customer specific resources they were supposed to connect to. Customer credentials and configuration were stored securely in AWS Secrets Manager. We connected **AWS Secrets Manager** with Kubernetes using **External Secrets**, allowing secrets to be synchronised into the appropriate Kubernetes namespaces without hardcoding credentials into the application or deployment configuration. This allowed each customer's services to automatically receive the configuration and credentials required to communicate with their corresponding resources.

AWS Infrastructure

The platform made extensive use of AWS services. Some of the infrastructure I worked with included:

- Amazon S3: frontend hosting, storage for indoor map/tile data and storing data backups
- Amazon CloudFront: CDN and caching for frontend content
- Amazon RDS: PostgreSQL databases
- AWS Secrets Manager: secure storage of customer credentials and configuration
- Kubernetes: application deployment and customer environment isolation

The Angular dashboard was initially deployed directly to an S3 bucket. As the platform evolved, we moved the dashboard deployment into our Kubernetes infrastructure. CloudFront was used as a CDN to cache content and improve retrieval speed for users accessing the application.

Database & Data Processing

The platform relied heavily on **PostgreSQL** because a significant part of the system's functionality depended on processing historical time series data. The database continuously collected data generated by the indoor localisation system. To ensure strong tenant isolation, we adopted a database per client architecture, providing each client with a dedicated database. This approach kept tenants' data completely isolated at the database level, reducing the risk of cross tenant data access and allowing data to be managed independently for each organisation.

A major part of my work involved improving database queries to make them both faster and more cost effective. Rather than only retrieving small datasets, the system needed to support queries over much larger time periods. I worked on queries capable of retrieving and processing time series information over periods of up to

approximately one month while maintaining practical response times. This required paying close attention to query structure, the amount of data being retrieved, and how the database processed the requested information.

CI/CD Pipeline

We followed a structured **CI/CD** pipeline using **GitHub**, **GitHub Actions**, and **Docker** to maintain code quality and ensure that changes were properly tested before being integrated into the main codebase. Development was carried out on forked repositories rather than directly on the main repository. Once a feature or change was completed, we created a **pull request** (PR) to propose merging the changes into the main repository.

Each pull request triggered a series of automated checks through GitHub Actions. These included code style and formatting checks, as well as automated tests to identify potential issues, such as syntax errors, invalid code, and other problems that could affect the application's functionality. This allowed us to identify issues early in the development process and ensure that proposed changes met the project's quality requirements before being integrated.

Following successful automated checks, the pull request was reviewed by another team member, who assessed the implementation and its compliance with the project's requirements and coding standards. Only after the necessary reviews and approvals was the pull request merged into the main repository.

Once the changes were merged, GitHub Actions automatically executed the repository's Dockerfile to build a Docker image of the application. The resulting image was then published to the **GitHub Container Registry** (GHCR), making it available for subsequent deployment to our Kubernetes cluster. This automated workflow provided a consistent process from development and code review through testing, containerisation, and preparation for deployment, while reducing manual intervention and helping maintain a reliable and reproducible software delivery process.

Problem Solving & System Development

My role gradually expanded beyond implementing individual frontend features. I became involved in understanding how the different parts of the system worked together from the devices deployed on construction sites, through the IoT communication layer and data import services, into PostgreSQL, and finally to the Angular dashboard used by customers. This gave me experience working across the complete data flow:

1. IoT Devices
2. The Things Stack
3. MQTT

4. Data Import
5. PostgreSQL
6. API Gateway
7. Angular Dashboard
8. Customer

Working across this flow also meant that investigating problems often required looking beyond a single application or service. For example, an issue visible in the dashboard could originate from the API, a database query, the data import service, the IoT communication layer, or the device itself. This experience strengthened my ability to approach problems from a systemwide perspective rather than focusing only on the part of the application where the problem initially appeared

Technology Stack

Frontend

- Angular
- TypeScript
- Mapbox
- Charting / data visualisation libraries

Backend

- Node.js
- REST APIs
- MQTT
- Webhooks
- Keycloak

Database

- PostgreSQL
- Amazon RDS
- Database migrations
- Time series data processing and query optimisation

Infrastructure & Cloud

- AWS
- Amazon S3
- Amazon CloudFront
- AWS Secrets Manager
- Kubernetes (AWS EKS)
- External Secrets

IoT

- The Things Stack

- MQTT
- Indoor localisation devices

Development & Deployment

- Git
- GitHub Actions
- Docker (for service testing)
- Kubernetes based deployments

Scope of My Role

Although I initially joined the company as a Frontend Developer, my responsibilities expanded significantly during my time there. I worked across the frontend, backend, database, authentication, cloud infrastructure, and deployment layers of the platform. This gave me practical experience not only in building individual features, but also in understanding how a production system is designed, deployed, secured, and operated across multiple services and customer environments.